TRYOLABS • MANUAL DE RESOLUCIÓN DE PROBLEMAS Y PREGUNTAS FRECUENTES EN VIVO

Qué Puede Salir Mal en los Laboratorios: 16 Preguntas de Asistentes y Soluciones Exactas

Mantén esta guía rápida a mano durante las 4 horas del taller de Google Antigravity (versiones Telecomunicaciones y Retail). Cada pregunta cubre un problema real que puede surgir en los Labs 01–04, junto con su causa raíz, qué responderle al participante y el comando exacto para resolverlo en segundos.

[Entorno e Instalación (Todos los Labs)] P1: 'Al ejecutar ./labs/lab-01-.../install.sh aparece python3: command not found, No module named venv, o un error de versión de Python.'

Causa Raíz: La laptop del asistente no tiene Python 3.10+ en el PATH o falta el paquete `python3-venv` en Linux/WSL.

Respuesta del Instructor y Solución Exacta:
1. Verifica si `uv` está instalado (`uv --version`). Si está disponible, ejecuta: `uv venv .venv --python 3.11 && source .venv/bin/activate && uv pip install -r requirements.txt`.
2. En macOS con Homebrew: `brew install python@3.11`.
3. En Windows: Ejecutar el script de PowerShell `.\labs\lab-01-...\install.ps1` en lugar de `.sh`.
4. Asegúrate de usar siempre `.venv/bin/python3` y `.venv/bin/pytest`.

[Entorno e Instalación (Todos los Labs)] P2: 'Al iniciar el dashboard FastAPI (uvicorn src.main:app --port 8000) falla con [Errno 48] Address already in use (macOS/Linux) o WinError 10048 (Windows).'

Causa Raíz: Otro proceso o una pestaña de terminal anterior ya está ocupando el puerto 8000 (o el puerto 8080 para `adk web`).

Respuesta del Instructor y Solución Exacta:
• En macOS/Linux, libera el puerto de inmediato: `lsof -ti:8000 | xargs kill -9` (o para el puerto 8080: `lsof -ti:8080 | xargs kill -9`).
• Alternativamente, levanta el servidor en otro puerto: `.venv/bin/uvicorn src.main:app --reload --port 8005`.

[Entorno e Instalación (Todos los Labs)] P3: 'Rompí accidentalmente la base de datos SQLite (data/*.db) o borré parte de un archivo probando cambios. ¿Cómo recupero mi entorno sin perder tiempo?'

Causa Raíz: El participante ejecutó una consulta destructiva o edición errónea durante las pruebas.

Respuesta del Instructor y Solución Exacta:
• Basta con volver a ejecutar el instalador del laboratorio actual: `./labs/lab-0X-.../install.sh`.
• Explica que cada script `install.sh` es **auto-recuperable e idempotente**: restaura el estado limpio del laboratorio actual, aplica las soluciones verificadas de los laboratorios previos y regenera la base de datos SQLite (`python3 data/seed_telecom_data.py` o `python3 data/seed_retail_data.py`) en 2 segundos.

[Entorno e Instalación (Todos los Labs)] P4: 'No llegué a terminar el Lab 1 a tiempo y ya vamos a empezar el Lab 2. ¿Me voy a quedar bloqueado?'

Causa Raíz: Diferencia normal de ritmo entre participantes durante un taller práctico.

Respuesta del Instructor y Solución Exacta:
• Tranquiliza al participante: '¡Para nada! Solo ejecuta `./labs/lab-02-rules-and-skills/install.sh`.'
• Cada `install.sh` aplica automáticamente las soluciones oficiales de los laboratorios anteriores antes de iniciar el nuevo laboratorio. Y si alguien quiere aplicar la solución completa del laboratorio actual, puede ejecutar `./labs/lab-0X-.../apply_solution.sh`.

[Autenticación Antigravity y Gemini Enterprise] P5: 'Antigravity muestra un error 403 PermissionDenied o indica que cloudaicompanion.googleapis.com no está habilitado para mi cuenta.'

Causa Raíz: El participante inició sesión con un correo personal de Gmail o falta asignarle el rol IAM en el proyecto de Google Cloud del taller.

Respuesta del Instructor y Solución Exacta:
1. Verifica que el asistente haya iniciado sesión en Antigravity con su cuenta corporativa asociada al taller.
2. En la consola de Google Cloud del instructor -> **IAM y Administración**, asigna al correo del asistente los roles `Gemini for Google Cloud User` (`roles/cloudaicompanion.user`) y `Service Usage Consumer` (`roles/serviceusage.serviceUsageConsumer`).
3. Confirma que la API `cloudaicompanion.googleapis.com` esté habilitada en **APIs y Servicios**.

[Lab 01 — Fórmulas y Lógica Telecom] P6 (Telecom): 'Antigravity editó bss_billing.py, ¡pero pytest sigue fallando en el monto del CDR por un 7.37% o por unos centavos!'

Causa Raíz: Dos errores clásicos en facturación BSS: (1) Dividir los bytes entre GB decimal (`10**9 = 1,000,000,000`) en lugar de GiB binario 3GPP (`2**30 = 1024**3 = 1,073,741,824`); o (2) Redondear `volume_gib` a 2 decimales *antes* de multiplicar por la tarifa y el multiplicador de roaming.

Respuesta del Instructor y Solución Exacta:
• Muestra la fórmula exacta: `volume_gib = (bytes_uplink + bytes_downlink) / 1073741824.0`.
• Mantén toda la precisión decimal durante la multiplicación (`charge = volume_gib * rate_per_gib * multiplier`) y aplica `round(charge, 2)` únicamente al monto final en dólares.
• Recuerda verificar `roaming_indicator != 'HOME'` (ej. `'ROAMING_INTL'`) para aplicar `roaming_markup_multiplier`.

[Lab 01 — Fórmulas y Lógica Telecom] P7 (Telecom): '¿Por qué la celda GNB-HOU-202 (slice URLLC) falla el test de SLA si su utilización PRB es de solo 64% (menor al 85%)?'

Causa Raíz: El participante asumió que una celda solo incumple el SLA cuando se cumplen AMBAS condiciones (`PRB > 85%` Y `latencia > límite`).

Respuesta del Instructor y Solución Exacta:
• Explica la realidad operativa en 5G: Un slice `URLLC` (`SST=2`) para cirugía robótica o redes eléctricas exige una latencia PDCP menor a `10ms`. Aunque el espectro PRB esté al 64%, la interferencia de radio (`SINR`) puede elevar la latencia a `18ms`, ¡incumpliendo el contrato SLA!
• En `src/services/ran_triage.py`, la condición debe usar `or`: `prb_utilization_pct > 85.0 or pdcp_latency_ms > sla_max_latency_ms`, ordenando descendentemente por `exposure_usd = sla_penalty_per_hour_usd * breach_duration_hours`.

[Lab 01 — Fórmulas y Lógica Retail] P8 (Retail): 'En margin_engine.py calculé el piso de 15% multiplicando unit_cost_usd * 1.15, ¿por qué pytest dice que el precio mínimo es incorrecto?'

Causa Raíz: Confusión clásica entre **15% de Recargo / Markup** (`costo * 1.15`) y **15% de Margen Bruto / Gross Margin** (`(precio - costo) / precio = 0.15`).

Respuesta del Instructor y Solución Exacta:
• Escribe ambas fórmulas en pantalla (es una aclaración comercial clave en retail):
- **Markup (Recargo sobre costo):** `precio = costo * 1.15` (si el costo es $85, `85 * 1.15 = $97.75`, ¡lo cual representa solo un 13.04% de margen bruto sobre el precio de venta!).
- **Gross Margin (Margen Bruto sobre venta):** `(precio - costo) / precio >= 0.15`, que despejando equivale a `min_price = round(unit_cost_usd / (1.0 - 0.15), 2)` (si el costo es $85, `$85 / 0.85 = $100.00`, dando exactamente 15.0% de margen bruto).
• Verifica también que los cupones `STACKABLE` se apliquen en cascada compuesta (`precio = precio * (1 - d1) * (1 - d2)`), nunca sumando porcentajes (`d1 + d2`).

[Lab 01 — Fórmulas y Lógica Retail] P9 (Retail): '¿Por qué mi validador GS1 GTIN-14 rechaza códigos de barras válidos de 14 dígitos en inventory_atp.py?'

Causa Raíz: Invertir los pesos `3` y `1` en el algoritmo Módulo-10 de GS1.

Respuesta del Instructor y Solución Exacta:
• En un código `GTIN-14` de 14 dígitos, los primeros 13 dígitos (`digits[:13]`) son los datos y el dígito 14 (`digits[13]`) es el dígito verificador.
• Recorriendo de izquierda a derecha los primeros 13 dígitos (índices `0` a `12`), los índices pares (`0, 2, 4, ... 12`) se multiplican por **`3`**, y los índices impares (`1, 3, 5, ... 11`) se multiplican por **`1`**. Luego `check_digit = (10 - (total % 10)) % 10`.

[Lab 02 — Rules y Skills Progresivos] P10: 'Creé el archivo SKILL.md dentro de .agents/skills/, ¿por qué Antigravity no descubre ni activa mi skill cuando se lo pido?'

Causa Raíz: El motor de divulgación progresiva depende estrictamente del frontmatter YAML al inicio de `SKILL.md` y de la ruta exacta.

Respuesta del Instructor y Solución Exacta:
Revisa estos 3 puntos con el asistente:
1. **Ruta y Mayúsculas Exactas:** Debe ser `.agents/skills/<nombre-skill>/SKILL.md` (`SKILL.md` en mayúsculas, dentro de `.agents/skills/`).
2. **Frontmatter YAML Válido:** La línea 1 de `SKILL.md` debe ser `---`, seguida por `name:` y `description:`, y cerrada con `---`.
3. **Palabras Clave en `description`:** Antigravity solo lee `name` y `description` inicialmente; asegúrate de que `description` mencione claramente cuándo activarse.

[Lab 02 — Rules y Skills Progresivos] P11: 'Cuando Antigravity ejecuta el script de mi skill (.agents/skills/.../scripts/run_audit.py), falla con ModuleNotFoundError: No module named src.'

Causa Raíz: Al ejecutar un script ubicado cuatro niveles dentro de `.agents/skills/<nombre>/scripts/`, Python no incluye automáticamente la raíz del repositorio en `sys.path`.

Respuesta del Instructor y Solución Exacta:
• Ejecuta el script anteponiendo `PYTHONPATH=. .venv/bin/python3 .agents/skills/.../scripts/run_audit.py`, O BIEN
• Agrega `sys.path.insert(0, str(Path(__file__).resolve().parents[4]))` al inicio de `run_audit.py` antes de importar `src.*`.

[Lab 03 — Servidor FastMCP y Hooks] P12: 'Mi servidor FastMCP funciona al importarlo en Python, pero Antigravity reporta error de conexión MCP o JSON-RPC inválido.'

Causa Raíz: La causa #1 de fallos en servidores MCP por `stdio`: ¡alguien puso un `print('Conectado...')` que escribe texto plano en `stdout`!

Respuesta del Instructor y Solución Exacta:
• Explica cómo funciona el transporte `stdio` en MCP: Antigravity lanza `mcp_server/*_mcp.py` como subproceso y se comunica por `stdin` y `stdout` usando exclusivamente mensajes JSON-RPC 2.0.
• Si cualquier línea hace `print()` en `stdout`, rompe el flujo JSON-RPC. Todo mensaje de log en un servidor MCP debe enviarse a `stderr`: `print(..., file=sys.stderr)`.
• Verifica también que `.agents/mcp_config.json` invoque a `.venv/bin/python3`.

[Lab 03 — Servidor FastMCP y Hooks] P13: 'Mi script de hook PreToolUse termina con sys.exit(1) ante una violación, ¿por qué Antigravity no lo trató como un bloqueo determinista de política?'

Causa Raíz: En la especificación de hooks de Antigravity, el código de salida `1` indica un error inesperado del script, mientras que el **código de salida `2`** es la señal explícita para bloquear determinísticamente la acción.

Respuesta del Instructor y Solución Exacta:
• Repasa el contrato de códigos de salida de Hooks en Antigravity:
- `sys.exit(0)` -> Acción permitida; continúa la ejecución de la herramienta.
- `sys.exit(2)` -> **¡Bloqueo Determinista!** Antigravity aborta la llamada a la herramienta de inmediato y envía el mensaje de `stderr` de vuelta al agente.
• Asegúrate además de registrar el evento en `logs/*.jsonl` antes de llamar a `sys.exit(2)`.

[Lab 04 — ADK, adk web, A2A y Terraform] P14: 'Al abrir ./scripts/run_adk_web.sh (http://localhost:8080) y enviar un mensaje en la interfaz ADK Dev UI, pide GOOGLE_API_KEY o credenciales de Vertex AI. ¿Cómo se configura?'

Causa Raíz: `pytest` y el endpoint FastAPI `/api/v1/a2a` incluyen un motor determinista local para que los tests pasen 100% offline, mientras que `adk web` conversa en vivo con el modelo Gemini dentro de `google.adk.agents.LlmAgent`.

Respuesta del Instructor y Solución Exacta:
• Para conversar en vivo con Gemini dentro de `adk web`, configura cualquiera de las dos opciones en la terminal antes de lanzar `./scripts/run_adk_web.sh`:
- **Opción A (Proyecto GCP de Vertex AI / Gemini Enterprise):**
`export GOOGLE_GENAI_USE_VERTEXAI=TRUE`
`export GOOGLE_CLOUD_PROJECT=<id-proyecto-gcp-del-taller>`
`export GOOGLE_CLOUD_LOCATION=us-central1`
`gcloud auth application-default login`
- **Opción B (Gemini API Key):**
`export GOOGLE_API_KEY=<tu-api-key>`
• Luego reinicia `./scripts/run_adk_web.sh` y abre `http://localhost:8080`.

[Lab 04 — ADK, adk web, A2A y Terraform] P15: 'En la interfaz ADK Dev UI (http://localhost:8080), el menú desplegable de agentes aparece vacío o no muestra telecom_nexus / retail_pulse.'

Causa Raíz: `adk web` requiere la estructura `adk_agents/<nombre_agente>/__init__.py` y `agent.py` exportando una variable llamada exactamente `root_agent`.

Respuesta del Instructor y Solución Exacta:
• Inicia siempre `adk web` mediante `./scripts/run_adk_web.sh` (que ejecuta `.venv/bin/adk web adk_agents --port 8080` desde la raíz del repositorio).
• Verifica que `adk_agents/<nombre_agente>/agent.py` defina `root_agent = LlmAgent(...)` y que `__init__.py` contenga `from . import agent`.

[Lab 04 — ADK, adk web, A2A y Terraform] P16: 'Al ejecutar ./deploy/deploy_gcp.sh para probar el despliegue Terraform, no tengo Docker Desktop corriendo o permisos de administrador GCP en mi laptop.'

Causa Raíz: El participante ejecutó `./deploy/deploy_gcp.sh` sin la bandera `--dry-run` en una laptop sin Docker o sin sesión activa de `gcloud`.

Respuesta del Instructor y Solución Exacta:
• Para validar el despliegue localmente en la laptop del participante, ejecuta `./deploy/deploy_gcp.sh --dry-run` (o `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run`).
• Explica que `--dry-run` verifica todos los archivos Terraform (`infra/terraform/main.tf`, `variables.tf`, `outputs.tf`), el `Dockerfile` y el script de registro en Gemini Enterprise 100% localmente sin requerir Docker ni credenciales de producción.